MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.3-4

Operating System

Checked by:

Reițman Petricia, *university assistant*

DISA, FCIM, UTM

Chișinău – 2026

Laboratory Work no. 3
The Command Line: Shell, Pipes and Text Filters

Purpose of the work

The purpose of this work is to learn to work fast in bash, to predict what the shell does with a command line before the command runs, to redirect the output where I want, and to answer questions about data with one line of small filters joined by pipes. At the end I put some of the one-liners into my first shell script.

Before you start

I ran the course setup script once, created the working folder `~/os-lab3` and generated the artificial web access log with the command from the task.

```
vboxuser@Ubuntu:~/os-lab3$ bash ~/Downloads/os2026-setup.sh
vboxuser@Ubuntu:~/os-lab3$ cd ~/os-lab3
vboxuser@Ubuntu:~/os-lab3$ for i in $(seq 1 500); do
> echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo
 FAIL || echo OK) /page$((i % 4))"
> done > access.log
vboxuser@Ubuntu:~/os-lab3$ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
vboxuser@Ubuntu:~/os-lab3$ ls
access.log
```

The check is correct: the last command printed `500 access.log`. Every line of the log has the date, the time, the user, the result (OK or FAIL) and the page.

Part 1. Move fast, find help

I typed `ls /usr/sh` and pressed Tab: bash completed it to `ls /usr/share/`. When I pressed Tab one more time, it asked if I want to see all the possible names. The same happened with `cd /et`: Tab completed it to `cd /etc/`, and one more Tab gave the question about the possibilities.

```
vboxuser@Ubuntu:~/os-lab3$ ls /usr/share/
Display all 253 possibilities? (y or n)
vboxuser@Ubuntu:~/os-lab3$ cd /etc/
Display all 140 possibilities? (y or n)
```

Then I ran three different commands (`whoami`, `pwd`, `ls`), pressed Ctrl+R and typed `wh`. Bash showed the old command `whoami` from the history:

```
vboxuser@Ubuntu:~/os-lab3$ whoami
vboxuser
```

```
vboxuser@Ubuntu:~/os-lab3$ pwd
/home/vboxuser/os-lab3
vboxuser@Ubuntu:~/os-lab3$ ls
access.log
(reverse-i-search)`wh': whoami
```

I checked which commands are built-in and where the shell looks for them:

```
vboxuser@Ubuntu:~/os-lab3$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
```

```
vboxuser@Ubuntu:~/os-lab3$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap
/bin:/snap/bin
vboxuser@Ubuntu:~/os-lab3$ echo $PATH | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
```

```
vboxuser@Ubuntu:~/os-lab3$ nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

I looked for help without a browser:

```
vboxuser@Ubuntu:~/os-lab3$ man ls
LS(1)                General Commands Manual                LS(1)

NAME
       ls - List directory contents.  Ignore files and directories starting with a '.' by
default
...
vboxuser@Ubuntu:~/os-lab3$ ls --help | head
List directory contents.
Ignore files and directories starting with a '.' by default

Usage: ls [OPTION]... [FILE]...
...
vboxuser@Ubuntu:~/os-lab3$ man -k "list directory"
dir (1)              - List directory contents.
gnuls (1)            - list directory contents
ls (1)               - List directory contents.
...
vboxuser@Ubuntu:~/os-lab3$ help cd
```

```
cd: cd [-L|[-P [-e]]] [-@] [dir]
    Change the shell working directory.
    ...
```

Observe (Part 1)

1. **Which of `cd`, `ls`, `echo` are built-ins, and why MUST `cd` be one?**
   `cd` and `echo` are shell built-ins (`type` says "is a shell builtin"). `ls` is not a built-in: it is an external program, the real file is `/usr/bin/ls`. In my shell `ls` is also an alias, so bash adds `--color=auto` every time I type `ls`. `cd` must be a built-in because the current directory belongs to the process. An external program is started as a child process: if `cd` was such a program, it would change the directory only of its own process and then exit, and my shell would stay in the old directory. So the shell has to do `cd` itself.
2. **How many directories are in the PATH, and in which one is `ls`?**
   `echo $PATH | tr ':' '\n'` printed 10 lines, so the PATH has 10 entries. One of them (`/snap/bin`) is written two times, so there are 9 different directories. `ls` is in `/usr/bin` (`which ls` printed `/usr/bin/ls`).
3. **What exit code does a command that is not found return?**
   It returns 127 (the message was `nosuchprog: command not found` and `exit code: 127`). A normal successful command returns 0.
4. **Which option of `ls` sorts by modification time?**
   The option `-t`: it sorts the files by modification time, the newest first. I found it in the manual of `ls` with `grep` (line 142 of the page says `-t Sort by modification time (the 'mtime' in the inode), newest first.`). Then I tested it with `ls -lt`: the newest file (`report.sh`) is at the top and the oldest (`access.log`) is at the bottom.

```
vboxuser@Ubuntu:~/os-lab3$ man ls | grep -n -i "modif"
142:       -t     Sort by modification time (the 'mtime' in the inode), newest first.
vboxuser@Ubuntu:~/os-lab3$ ls -lt
total 52
-rwxrwxr-x 1 vboxuser vboxuser   303 Oct 12 14:24 report.sh
-rw-rw-r-- 1 vboxuser vboxuser  1031 Oct 12 14:15 oneliners.sh
-rw-rw-r-- 1 vboxuser vboxuser  2288 Oct 12 13:36 all.txt
drwxrwxr-x 2 vboxuser vboxuser  4096 Oct 12 13:34 tmp
-rw-rw-r-- 1 vboxuser vboxuser    13 Oct 12 13:31 notes.md
-rw-rw-r-- 1 vboxuser vboxuser    53 Oct 12 13:30 err.txt
-rw-rw-r-- 1 vboxuser vboxuser  2288 Oct 12 13:30 out.txt
drwxrwxr-x 2 vboxuser vboxuser  4096 Oct 12 13:02 play
-rw-rw-r-- 1 vboxuser vboxuser 16700 Oct 12 11:12 access.log
```

Part 2. What the shell does to your line

I created a test directory with a few files:

```
vboxuser@Ubuntu:~/os-lab3$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
vboxuser@Ubuntu:~/os-lab3/play$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt";
```

```
ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Before running each line, I wrote down what I think it will print. The table has my prediction, the real output and an explanation if they were different.

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	all .c files	`a.c b.c`	
`echo file?.txt`	file + any one character + .txt	`file1.txt file2.txt`	
`echo [a-b]*`	all files starting with a or b	`a.c alfa beta.txt b.c`	
`echo *.xyz`	no such files	`*.xyz`	No file matches the pattern, so bash leaves it unexpanded and passes it to `echo` as plain text.
`ls alfa beta.txt`	No such file (2 nonexistent arguments)	`ls: cannot access 'alfa': No such file or directory` and `ls: cannot access 'beta.txt': No such file or directory`	
`ls "alfa beta.txt"`	same file	`'alfa beta.txt'`	
`echo '$HOME' "$HOME" \$HOME`	main folder	`$HOME /home/vboxuser $HOME`	Only the middle argument is the home folder. Single quotes disable expansion, double quotes allow `$` expansion, and the backslash escapes `$`.
`echo {1..5} ; echo $((7 * 6))`	1 2 3 ... and 42	`1 2 3 4 5` and `42`	
`echo "today is $(date +%A), I am $(whoami)"`	message with today's day and user name	`today is Monday, I am vboxuser`	
`touch log{1..3}.txt && ls log*`	creates log1.txt, log2.txt, log3.txt and lists them	`log1.txt log2.txt log3.txt`	

Globbing results:

```
vboxuser@Ubuntu:~/os-lab3/play$ echo *.c
a.c b.c
vboxuser@Ubuntu:~/os-lab3/play$ echo file?.txt
file1.txt file2.txt
vboxuser@Ubuntu:~/os-lab3/play$ echo [a-b]*
a.c alfa beta.txt b.c
vboxuser@Ubuntu:~/os-lab3/play$ echo *.xyz
*.xyz
```

The name with a space:

```
vboxuser@Ubuntu:~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
vboxuser@Ubuntu:~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
```

Quotes and backslash:

```
vboxuser@Ubuntu:~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/vboxuser $HOME
```

The other lines:

```
vboxuser@Ubuntu:~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
vboxuser@Ubuntu:~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am vboxuser
vboxuser@Ubuntu:~/os-lab3/play$ touch log{1..3}.txt && ls log*
log1.txt log2.txt log3.txt
```

Observe (Part 2)

1. **For each line: prediction, real output and one sentence if they differ.**
   See the table above. All my predictions were right in the idea. The only line that needs an explanation is `echo *.xyz`: I expected nothing, but bash printed the pattern itself, because when no file matches, bash leaves the pattern as it is.
2. **`echo` knows nothing about files, yet `echo *.c` printed file names. Who produced them?**
   The shell did it. Before running `echo`, bash looks at the line, sees the pattern `*.c`, replaces it with the names of the matching files (`a.c b.c`) and only then starts `echo` with these two words as arguments. So `echo` just printed what it received. `{1..5}`, `$((7 * 6))`, `$(date +%A)` and `$HOME` are also replaced by the shell.
3. **Why did `ls alfa beta.txt` fail? Two ways to fix it.**
   The shell splits the line into words at the spaces, so `ls` got two arguments, `alfa` and `beta.txt`, and neither of these files exists. The file is really called `alfa beta.txt` (one name with a space inside). Two ways to fix it: put the name in quotes, `ls "alfa beta.txt"` (this one I ran and it worked), or put a backslash before the space, `ls alfa\ beta.txt`.
4. **What is the difference between single and double quotes?**
   Single quotes keep everything as plain text: nothing inside is expanded, so `'$HOME'` stays `$HOME`. Double quotes keep the spaces together, but still allow expansion with `$`, so `"$HOME"` became `/home/vboxuser`. The backslash takes away the special meaning of only one next character (`\$HOME` stayed `$HOME`).
5. **Why does `ls` show `'alfa beta.txt'` in quotes?**
   Because the name has a space. `ls` adds the quotes to show that it is one name and not two files, and

so I can copy it into a command and it will work.

Part 3. Redirection, chaining and jobs

I redirected stdout and stderr to different files:

```
vboxuser@Ubuntu:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
vboxuser@Ubuntu:~/os-lab3$ wc -l out.txt ; cat err.txt
238 out.txt
ls: cannot access '/nope': No such file or directory
```

```
vboxuser@Ubuntu:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh_command_not_found
vboxuser@Ubuntu:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
vboxuser@Ubuntu:~/os-lab3$ wc -l < access.log
500
```

I compared the order of the redirections:

```
vboxuser@Ubuntu:~/os-lab3$ ls /etc /nope > all.txt 2>&1
vboxuser@Ubuntu:~/os-lab3$ head -2 all.txt
ls: cannot access '/nope': No such file or directory
/etc:
vboxuser@Ubuntu:~/os-lab3$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
vboxuser@Ubuntu:~/os-lab3$ head -2 all.txt
/etc:
ModemManager
```

I tried the operators `;`, `&&` and `||`:

```
vboxuser@Ubuntu:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
vboxuser@Ubuntu:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/vboxuser/os-lab3/tmp
```

I started a background job and stopped it:

```
vboxuser@Ubuntu:~/os-lab3$ sleep 100 &
[1] 21770
vboxuser@Ubuntu:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
```

The second `jobs` still showed `Running`, because it ran immediately after `kill` and the shell had not yet noticed that the process ended. The message `Terminated` appeared right after.

Observe (Part 3)

1. **Which file got the error message, and which got the listing?**
   In `ls /etc /nope > out.txt 2> err.txt` the listing of `/etc` went to `out.txt` (238 lines) and the error message `cannot access '/nope'` went to `err.txt`. Stream 1 (stdout) is redirected with > and stream 2 (stderr) with `2>`.
2. **What is the difference between `> all.txt 2>&1` and `2>&1 > all.txt`?**
   The shell reads the redirections from left to right.
   - In `> all.txt 2>&1`: first stdout goes to the file, then stderr is sent to the same place where stdout goes now, which is the file. So both the error and the listing are in `all.txt`.
   - In `2>&1 > all.txt`: first stderr is sent to the place where stdout goes now, which is still the terminal, and only after that stdout is moved to the file. So the error was printed on the screen, and only the listing is in `all.txt`.
3. **Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the file name?**
   With < the shell opens the file and gives it to `wc` as its standard input, so `wc` only sees a stream of text and does not know any file name. With `wc -l access.log` the name is an argument, `wc` opens the file itself and prints the name next to the number.
4. **The operators `;`, `&&`, `||` in one sentence each.**
   - `;` runs the next command always, when the first one is finished (it does not matter if it worked or not).
   - `&&` runs the next command only if the first one succeeded (exit code 0).
   - `||` runs the next command only if the first one failed (exit code not 0), like in `ls /nope 2> /dev/null || echo "failed with code $?"`, which printed `failed with code 2`.

Part 4. Answer questions with one-liners

I answered every question with one command line and saved all of them in the file `oneliners.sh`.

```
vboxuser@Ubuntu:~/os-lab3$ cat oneliners.sh
# Q1. How many requests failed?
grep -c FAIL access.log
# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | tee /dev/stderr | wc -l
# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c
# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1} $1==m'
# Q5. How many requests did user3 make, and how many of them failed?
grep ' user3 ' access.log | cut -d' ' -f4 | sort | uniq -c
# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
```

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
ls /etc | wc -l ; ls -l /etc | wc -l
# Bonus. Lines where the minute ends in 0 and the request failed.
grep -E '10:[0-5]0 user[0-9] FAIL' access.log | wc -l
```

Q1. How many requests failed? `grep -c` counts the lines that have the word FAIL: 100 requests failed.

```
vboxuser@Ubuntu:~/os-lab3$ grep -c FAIL access.log
100
```

Q2. How many different pages were requested, and which? `cut` takes the 5th field (the page), `sort -u` leaves every page once. `tee /dev/stderr` shows the list of pages on the screen and lets the same list go on to `wc -l`, which counts them. There are 4 different pages.

```
vboxuser@Ubuntu:~/os-lab3$ cut -d' ' -f5 access.log | sort -u | tee /dev/stderr | wc -l
/page0
/page1
/page2
/page3
4
```

Q3. How many requests did each page get? Check: $4 \times 125 = 500$, so the counts add up to the number of lines of the log.

```
vboxuser@Ubuntu:~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Q4. Which user or users have the most failed requests, and how many? Two users have the most failures, `user3` and `user5`, 15 each. I used `awk` to keep every line whose count is equal to the count of the first line, so a tie is shown too. Check: the failures of all users from `report.sh` are $14 + 14 + 14 + 15 + 14 + 15 + 14 = 100$, the same as Q1.

```
vboxuser@Ubuntu:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn |
awk 'NR==1{m=$1} $1==m'
     15 user5
     15 user3
```

Q5. How many requests did user3 make, and how many of them failed? `user3` made $15+57 = 72$ requests, and 15 of them failed.

```
vboxuser@Ubuntu:~/os-lab3$ grep ' user3 ' access.log | cut -d' ' -f4 | sort | uniq -c
     15 FAIL
     57 OK
```

Q6. Print the last 3 failed requests, showing only time and user.

```
vboxuser@Ubuntu:~/os-lab3$ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
```

```
10:15 user5
10:20 user3
```

Q7. Which login shells appear in /etc/passwd, and how many accounts use each? The shell is the 7th field and the separator in `/etc/passwd` is `:`. Most accounts (40) are system accounts with `nologin`, and only 2 accounts use `/bin/bash`.

```
vboxuser@Ubuntu:~/os-lab3$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
```

Q8. Why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one? My prediction: `ls -l /etc | wc -l` will be bigger because of the `total` line at the top.

```
vboxuser@Ubuntu:~/os-lab3$ ls /etc | wc -l ; ls -l /etc | wc -l
237
238
```

`ls /etc` prints one name per line when the output goes into a pipe, so `wc -l` counts 237 entries. `ls -l` prints the same 237 lines, one for each entry, and one more line at the top, `total ...`, with the size of the files in blocks. That is why the second number is bigger by exactly one. I checked it by looking at the first lines of `ls -l /etc`:

```
vboxuser@Ubuntu:~/os-lab3$ ls -l /etc | head -3
total 1156
drwxr-xr-x  5 root                root                  4096 Aug 26 05:30 ModemManager
drwxr-xr-x  8 root                root                  4096 Sep 29 16:30 NetworkManager
```

Bonus. Lines where the minute ends in 0 and the request failed. There are 50 such lines. It is also logical: the minute ends in 0 when the line number is a multiple of 10, and then the number is also a multiple of 5, so the request is always a FAIL. Between 1 and 500 there are 50 multiples of 10.

```
vboxuser@Ubuntu:~/os-lab3$ grep -E '10:[0-5]0 user[0-9] FAIL' access.log | wc -l
50
```

Full run of `oneliners.sh`:

```
vboxuser@Ubuntu:~/os-lab3$ bash oneliners.sh
100
/page0
/page1
/page2
/page3
4
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
     15 FAIL
```

```
     57 OK
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
237
238
50
```

Part 5. Your first script

I created `report.sh` and completed the TODO with a loop over the users from `user0` to `user6`.

```
vboxuser@Ubuntu:~/os-lab3$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
echo "no such file: $log" >&2
exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
echo "user$n: $(grep FAIL "$log" | grep -c "user$n ") failures"
done
```

I tried to run it before `chmod +x`, and then after it:

```
vboxuser@Ubuntu:~/os-lab3$ ./report.sh access.log
bash: ./report.sh: Permission denied
vboxuser@Ubuntu:~/os-lab3$ chmod +x report.sh
vboxuser@Ubuntu:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

```
vboxuser@Ubuntu:~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Observe (Part 5)

1. **What does the first line `#!/bin/bash` do, and what happens if I run `./report.sh` before `chmod +x`?**
   The first line (it is called shebang) tells the system which program must run the file: here it is `/bin/bash`. Before `chmod +x` the file did not have the permission to execute, so the shell refused to start it and printed `bash: ./report.sh: Permission denied`. After `chmod +x` it worked.
2. **Why must I type `./report.sh` and not just `report.sh`?**
   When I type a name without a slash, the shell looks for the program only in the directories of the `PATH`. As I saw in Part 1, the current directory is not in my `PATH`, so `report.sh` would give `command not found`. With `./` I give the path to the file myself (the dot means the current directory), so the shell does not need to search.
3. **Why does the script write the error with `>&2` and end with `exit 1`?**
   `>&2` sends the message to stderr (stream 2) and not to stdout, so the error can be separated from the normal output, for example with `2> err.txt`, like in Part 3. `exit 1` ends the script with a non-zero exit code. The exit code 0 means success and everything else means a problem. Other commands and scripts check it with `$?`, `&&` and `||`, so they know that the script failed (in my test `echo "exit code: $?"` printed 1).

Conclusion (Lab 3)

In this work I learned to use the command line faster: Tab completion, the history search with Ctrl+R, and the commands `type`, `which`, `man` and `help` to find where a command comes from and how to use it. I saw that the shell does a lot of work before a program starts: it expands `*`, `?`, `[a-b]`, `{1..5}`, `$HOME` and `$(...)`, and it splits the line at the spaces, which is why I need quotes for `alfa beta.txt`. I redirected stdout and stderr to different files and understood that the order of `> file` and `2>&1` is important, because the shell reads it from left to right. With pipes and the filters `grep`, `cut`, `sort`, `uniq`, `wc`, `head` and `tail` I answered eight questions about the log of 500 lines, and checked the numbers (the counts of the pages add up to 500, and the failures of all users add up to 100). At the end I wrote my first script `report.sh`, and learned about the shebang line, the execute permission, `./`, stderr and exit codes.

Laboratory Work no. 4
The xv6 Shell from the Outside

Purpose of the work

The purpose of this work is to see what a shell really has to do and what a bigger shell like bash adds on top. I use the xv6 shell and bash side by side and compare them. I also write a small program that prints its arguments, to see who expands `*` and who removes the quotes. Before that I make sure that my `sleep` program builds on the current xv6.

Part 1. Fix sleep for the current xv6

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`, so my `sleep` program from Lab 2 may not build any more. First I checked which name my xv6 tree uses.

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

`user.h` declares `pause`, so I had to change the call in `user/sleep.c`. This is the updated program:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Then I checked that `_sleep` is in `UPROGS` in the Makefile:

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "_sleep" Makefile
135:        $U/_sleep\
```

After that I built and started xv6 with `make qemu`. The build finished without errors (`user/_sleep` was linked and added to `fs.img`) and xv6 booted. Then I tested the program:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sleep 20
```

```
$ sleep
usage: sleep <ticks>
$
```

Result: `sleep 20` waits 20 ticks and prints nothing, and `sleep` without an argument prints the usage message.

Part 2. bash versus the xv6 shell

I ran every command twice: in bash on Linux (in an empty test folder `~/shtest` that has only a file `README`) and at the xv6 `$` prompt.

Commands run in bash:

```
vboxuser@Ubuntu:~$ rm -rf ~/shtest
vboxuser@Ubuntu:~$ mkdir ~/shtest
vboxuser@Ubuntu:~$ cd ~/shtest
vboxuser@Ubuntu:~/shtest$ touch README
vboxuser@Ubuntu:~/shtest$ echo hello > f
vboxuser@Ubuntu:~/shtest$ cat < f
hello
vboxuser@Ubuntu:~/shtest$ ls | grep READ
README
vboxuser@Ubuntu:~/shtest$ (echo a; echo b) | wc
      2       2       4
vboxuser@Ubuntu:~/shtest$ echo one
one
vboxuser@Ubuntu:~/shtest$ echo two
two
vboxuser@Ubuntu:~/shtest$ echo bg & wait
bg
vboxuser@Ubuntu:~/shtest$ ls *
README  f
vboxuser@Ubuntu:~/shtest$ echo $HOME
/home/vboxuser
vboxuser@Ubuntu:~/shtest$ echo "a b"
a b
vboxuser@Ubuntu:~/shtest$ mkdir d
vboxuser@Ubuntu:~/shtest$ cd d
vboxuser@Ubuntu:~/shtest/d$ ls ..
README  d  f
```

Here every command is typed on its own line. I first ran all of them in one line with `bash -x`, and then bash also printed every command after the expansion (the lines with +). For example `ls *` became `+ ls f README` and `echo $HOME` became `+ echo /home/vboxuser`, so I can see what the shell really gave to the program.

The same commands run in xv6:

```
$ echo hello > f
$ cat < f
hello
$ ls | grep READ
README          2 2 2441
```

```
$ (echo a; echo b) | wc
2 2 4
$ echo one
one
$ echo two
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d
$ cd d
$ ls ..
exec ls failed
```

To run `ls` after `cd d` I have to give the path to the program, for example `/ls ..`:

```
$ cd d
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
sleep          2 5 35280
forktest       2 6 18136
grep           2 7 44184
init           2 8 35976
kill           2 9 35512
ln             2 10 35304
ls             2 11 42880
mkdir          2 12 35568
rm             2 13 35552
sh             2 14 58312
stressfs       2 15 36416
usertests      2 16 206800
grind          2 17 51824
wc             2 18 37728
zombie         2 19 34848
logstress      2 20 37560
forphan        2 21 36352
dorphan        2 22 35800
sync           2 23 34944
console        3 24 0
f              2 25 6
d              1 26 32
$
```

It works, and the list shows that all the programs (`cat`, `echo`, `ls`, `sleep`, `sh` …) are in `/`, and

the folder `d` and the file `f` are there too.

Comparison table:

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	`hello`	`hello`	< and > are done by the shell itself: it opens the file and puts it in place of stdin/stdout before running the program.
`ls | grep READ`	pipe	`README`	`README 2 2 2441`	Both shells have pipes. In xv6 `ls` prints more (type, inode number and size), so the line looks different.
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`	Both shells understand `( )` and `;`.
`echo one ; echo two`	sequence	`one` and `two`	`one` and `two`	`;` runs the commands one after another in both shells.
`echo bg &`	background	`bg`	`bg`	`&` starts the command and the shell does not wait for it.
`ls *`	globbing	`README f`	`ls: cannot open *`	bash replaces `*` with the file names. The xv6 shell does not, so `ls` gets a real `*` and there is no file with this name.
`echo $HOME`	variables	`/home/vboxuser`	`$HOME`	xv6 has no environment variables, so the text stays as it is.
`echo "a b"`	quotes	`a b`	`"a b"`	bash removes the quotes. xv6 keeps them in the text.
`mkdir d ; cd d ; ls ..`	PATH	`README d f`	`exec ls failed`	xv6 has no PATH. It looks for the file `ls` in the current folder `d`, and it is not there.
arrow up, Tab, Ctrl+R	history, completion	all three work: arrow up brought back `ls README`, Tab completed `ls RE` to `ls README`, Ctrl+R found the old command `(reverse-i-search)`	none of them work	bash uses a line editor (readline) that remembers the commands and completes the names. The xv6 shell only reads a line of text from the keyboard, so these keys do nothing useful.

Observe (Part 2)

1. **Features that both shells have and features that only bash has.**
   Both shells can run programs, redirect input and output with < and >, make pipes with `|`, run com-

mands in a sequence with `;`, group commands with `( )` and run a command in the background with `&`. Only bash has globbing (`*`), variables (`$HOME`), quotes, `PATH`, command history (arrow up), Tab completion and Ctrl+R search. Bash also has other things that I did not test here, for example `&&`, `||` and scripts.

2. **Why `ls` stops working after `cd d`, and how to run it anyway.**
   xv6 has no `PATH`. When I type `ls`, the shell calls `exec("ls", ...)`, and `exec` looks for a file called `ls` in the current folder. All xv6 programs live in the root folder `/`. After `cd d` I am in `d`, where there is no `ls` file, so I get `exec ls failed`. To run it anyway I have to give the path to the program, for example `/ls ..` (see the output above).

3. **Which missing feature I would miss most, and which is the easiest to add.**
   I would miss the command history (arrow up) the most, because without it I have to type every command again. The easiest to add is the search in `PATH`, I think: when `exec` does not find the command, the shell could try again with `/` in front of the name. It needs only a few lines in the shell. Globbing is harder, because the shell would have to read the folder and compare the names with the pattern, and variables are harder still, because they need a whole place to store them.

Part 3. Who expands *? A program that prints its argv

`user/args.c` for xv6, with its line in `UPROGS` in the Makefile:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "_args" Makefile
133:        $U/_args\
```

The same program for Linux, `args_linux.c` (it uses `<stdio.h>` and `return 0`):

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
```

```
}
```

I compiled it in a folder `~/shtest3` that has the files `a.c`, `b.c` and `c.c`:

```
vboxuser@Ubuntu:~$ rm -rf ~/shtest3
vboxuser@Ubuntu:~$ mkdir ~/shtest3
vboxuser@Ubuntu:~$ cd ~/shtest3
vboxuser@Ubuntu:~/shtest3$ touch a.c b.c c.c
vboxuser@Ubuntu:~/shtest3$ gcc -o args ~/args_linux.c
```

Output in xv6:

```
$ args one   two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Output in bash:

```
vboxuser@Ubuntu:~/shtest3$ ./args one   two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
vboxuser@Ubuntu:~/shtest3$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [b.c]
argv[3] = [c.c]
argv[4] = [x y]
vboxuser@Ubuntu:~/shtest3$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vboxuser]
vboxuser@Ubuntu:~/shtest3$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
```

```
argv[1] = [single quoted]
```

Side by side, the line `args *.c "x y"` gave this:

	bash	**xv6**
argc	5	4
arguments	`./args`, `a.c`, `b.c`, `c.c`, `x y`	`args`, `*.c`, `"x`, `y"`

Observe (Part 3)

1. **How many arguments did each program receive for `args *.c "x y"`?**
   The program in bash received 5 arguments (`argc = 5`): its own name, `a.c`, `b.c`, `c.c` and `x y`. The program in xv6 received 4 (`argc = 4`): its own name, `*.c`, `"x` and `y"`. The first argument `argv[0]` is always the name of the program, so it is also counted.
2. **Who turns `*.c` into file names and removes the quotes: the program, the kernel or the shell?**
   It is the shell. The program gets the ready array `argv` and only prints it, and the kernel only passes this array to the program when `exec` is called. The proof is the difference between the two outputs: it is the same program idea, but in xv6 the characters `*.c`, `"x` and `$HOME` arrive without any change, because the xv6 shell cannot expand them. In bash, when I ran the lines with `bash -x`, bash printed `+ ./args a.c b.c c.c 'x y'`, and this shows that the expansion was already done before `args` was started.
3. **What happened to the extra spaces in `one  two  three`?**
   They disappeared in both shells. The shell uses spaces only to separate the words, and several spaces in a row work like one. So `argv` has three clean words (`one`, `two`, `three`) and `argc = 4`, because the program name is counted too. To keep spaces inside one argument I need quotes, but in xv6 the quotes do not work.

Part 4. Read the shell

I ran these commands inside `~/xv6-riscv` and read the places in `user/sh.c` that they show.

```
vboxuser@Ubuntu:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

The file has 499 lines, which fits the “about 500 lines” from the task.

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
```

```
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

The lines 75 to 125 are in the function `runcmd`, so this is where each kind of command is run. The lines 284 to 294 are the characters that the parser treats as special, and the lines 469 to 493 are another `switch` over the same five kinds.

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

The search found only the error message, because the check for `cd` is written with single characters (`cmd[0] == 'c' && cmd[1] == 'd'`), so the word `cd` is not in one piece there. That is why I printed the lines around it:

```
vboxuser@Ubuntu:~/xv6-riscv$ sed -n '160,180p' user/sh.c
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
  }
  exit(0);
}
```

Here the shell checks if the line starts with `cd` and then calls `chdir` itself. Only if it is not `cd`, it calls `fork1()` and runs the command in the child with `runcmd`.

I also searched `sh.c` for `*` and `$` with a `grep -n` command, and it printed nothing.

Observe (Part 4)

1. **The five kinds of command and the character that produces each.**
   - `EXEC`: a simple command with arguments, just words separated by spaces.
   - `REDIR`: redirection of input or output, <, > (and >>).
   - `PIPE`: a pipe between two commands, `|`.
   - `LIST`: commands one after another, `;`.
   - `BACK`: a command in the background, `&`.

   The brackets `( )` are not a sixth kind. They only group commands when the parser reads the line.
2. **Where is `cd` handled, and why is it not a separate program?**
   `cd` is handled in the shell itself, in the main loop of `main` that reads the lines (`while (getcmd(...))`, line 160). The check `cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '` is on line 166, and `chdir(cmd + 3)` is called right there, in the shell process. Only the `else` branch calls `fork1()`. The comment in the code says it directly: "Chdir must be called by the parent, not the child". The reason is that the current folder belongs to a process. If `cd` was a separate program, the shell would start it as a child process, the child would change its own folder and exit, and the shell would stay in the old folder. So the shell has to call `chdir` itself.
3. **Is there code for `*` or `$` in `sh.c`? What does it explain about Part 2?**
   No, the search found nothing. The special characters of the parser are only `|`, `(`, `)`, `;`, `&`, < and > (lines 286–294), and + for >> (line 397). This explains Part 2 and Part 3: in xv6 `ls *` looks for a file that is really named `*`, and `$HOME` stays the text `$HOME`, because the shell has no code that can expand them.

Conclusion (Lab 4)

In this work I used the xv6 shell and bash side by side. First I fixed my `sleep` program for the new name of the system call (`pause` instead of `sleep`) and checked it in xv6. Then I compared the two shells and saw that the xv6 shell can do the basic things (programs, redirection, pipes, `;`, `( )` and `&`), but it has no globbing, variables, quotes, `PATH` and history. With my `args` program I found out that the expansion of `*` and the removing of quotes are done by the shell, not by the program or the kernel: the kernel only gives `argv` to the program. At the end I read `user/sh.c` (499 lines) and saw the five kinds of commands (`EXEC`, `REDIR`, `PIPE`, `LIST`, `BACK`) and that `cd` must be done by the shell itself, because it changes the folder of the shell process. So a shell is quite a small program, and a lot of what we are used to in bash is an extra on top of it.